

SGX Admin Guide

Project Name: SGX (Cervais)

Version: Phase 1

Date: 27 Jan, 2025

Introduction:

The SG-X Guardian Client is a Zero-Trust Edge Security Agent designed for robust protection of distributed IoT and enterprise nodes. It ensures continuous security by operating autonomously, enforcing policies, performing and maintaining secure peer-to-peer communication for critical functions that persist even during network outages or cloud disconnection.

SGX provides a enforced Trusted Execution Environment that protects SG-X Guardian's most critical security operations from the host operating system and other privileged software. It ensures cryptographic keys, policy enforcement logic, and security intelligence remain confidential and tamper-resistant even if the device is compromised. SGX enables remote attestation, allowing Guardian to prove its integrity and authenticity in untrusted or hostile environments.

Installation & Project Setup

Rust Toolchain (MANDATORY – rustup only)

APT Rust strictly forbidden (unstable + outdated)

```
Install required Rust version
rustup toolchain install 1.90.0
rustup default 1.90.0
```

Verify:

```
rustc --version
cargo --version
```

Expected:

```
rustc 1.90.0
cargo 1.90.0
```

Fix PATH issues (MOST COMMON FAILURE)

Check rustc path:

```
which rustc
```

✗ Wrong:

```
/usr/bin/rustc
```

Correct:

```
~/.cargo/bin/rustc
```

Temporary fix (test):

```
export PATH="$HOME/.cargo/bin:$PATH"
```

Permanent fix:

```
nano ~/.bashrc
```

Add at END:

```
export PATH="$HOME/.cargo/bin:$PATH"
```

Reload:

```
source ~/.bashrc
```

Remove system Rust (STRONGLY RECOMMENDED)

```
sudo apt remove -y rustc cargo  
sudo apt autoremove -y
```

Reason:

SGX Guardian requires deterministic builds — system Rust breaks CI parity.

Protobuf Compiler (protoc 33.0)

Remove old versions (if any)

```
which protoc  
Install protoc 33.0  
cd /tmp  
rm -rf protoc-33.0*  
uname -m
```

For x86_64:

```
wget https://github.com/protocolbuffers/protobuf/releases/download/v33.0/protoc-33.0-linux-x86_64.zip  
unzip protoc-33.0-linux-x86_64.zip -d protoc-33.0
```

Install binary:

```
sudo cp protoc-33.0/bin/protoc /usr/local/bin/protoc  
sudo chmod +x /usr/local/bin/protoc
```

Install includes:

```
sudo mkdir -p /usr/local/include/google  
sudo rsync -av protoc-33.0/include/google/ /usr/local/include/google/
```

Verify:

```
which protoc  
protoc --version  
hash -r
```

Expected:

```
libprotoc 33.0
```

Project Rust Version Lock (IMPORTANT)

Inside project root:

```
cd ~/SGX  
rustup override set 1.90.0
```

Verify:

```
rustup show
```

Expected:

```
active toolchain: 1.90.0 (override)
```

Clean Build Test (Sanity Check)

```
cargo clean  
cargo build
```

If this passes → environment READY.

SG-X Guardian — Admin & Developer Guide

Goal

Generate keys once, sign policy, place it correctly, then start Guardian nodes.
No duplicate key generation. No ambiguity.

System Initialization (Filesystem):

```
sudo mkdir -p /etc/sgx-guardian/schemas  
sudo mkdir -p /etc/sgx-guardian/policies  
sudo mkdir -p /etc/sgx-guardian/config  
sudo mkdir -p /var/lib/sgx-guardian  
sudo mkdir -p /var/log/sgx-guardian
```

Install Policy Schema & Node Configs:

```
sudo cp schemas/uep_policy_v1.yaml /etc/sgx-guardian/schemas/  
sudo cp config/nodeA.yaml /etc/sgx-guardian/config/  
sudo cp config/nodeB.yaml /etc/sgx-guardian/config/  
sudo cp config/nodeC.yaml /etc/sgx-guardian/config/  
sudo chown -R root:root /etc/sgx-guardian  
sudo chmod -R 755 /etc/sgx-guardian
```

Key Generation

This step must be done once per host, not per role.

From cd sgx-pa-cli:

```
sudo env "PATH=$PATH" "HOME=$HOME" cargo run -- keygen
```

✓ Generates:
guardian_private.key
guardian_public.key

Install them:

```
sudo cp guardian_private.key /etc/sgx-guardian/  
sudo cp guardian_public.key /etc/sgx-guardian/  
sudo chmod 600 /etc/sgx-guardian/guardian_private.key  
sudo chmod 644 /etc/sgx-guardian/guardian_public.key
```

Policy Signing

Uses the same keys generated above:

```
cargo run -- sign /etc/sgx-guardian/schemas/uep_policy_v1.yaml
```

✓ Output:

```
policy.sig
```

Install policy:

```
sudo mv policy.sig /etc/sgx-guardian/policies/policy.sig  
sudo chmod 644 /etc/sgx-guardian/policies/policy.sig
```

Policy Verification (Before Node Start)

```
cargo run -- verify --signed /etc/sgx-guardian/policies/policy.sig
```

Expected:

- ✓ Envelope version valid
- ✓ Digest valid
- ✓ Signature valid
- ✓ Policy integrity verified successfully
- ✗ If this fails → STOP

Start Guardian Nodes

Node A

```
sudo env "PATH=$PATH" "HOME=$HOME" cargo run -- nodeA
```

Node B

```
sudo env "PATH=$PATH" "HOME=$HOME" cargo run -- nodeB
```

Node C

```
sudo env "PATH=$PATH" "HOME=$HOME" cargo run -- nodeC
```

Expected

```
Policy loaded at startup
Policy digest bound to attestation
Nodes only trust same-policy peers
```

Post-Boot Validation

```
cd sgx-pa-cli
cargo run -- peers
cargo run -- attestation
```

SG-X Guardian — Admin & Developer Guide

Scope

This section documents the exact runtime flow that works in practice:

- Primary Guardian runs via systemd (.deb)
- Additional nodes run via cargo
- nftables verified after runtime is active
- Logs and audit validated at the end
 - No policy signing here
 - No key generation here
 - No duplicate node starts

Entry Preconditions

- Policy is signed, verified, and placed
- Keys already exist
- .deb package is installed
- systemd service is available

1. Start Primary Guardian (Node-A via systemd)

This step is done ONLY ONCE

```
sudo systemctl start sgx-guardian
```

Check status:

```
sudo systemctl status sgx-guardian
```

Expected (important line):

```
Active: active (running)
```

This confirms:

- Primary Guardian (Node-A) is running
- Ports are already bound
- Enforcement mode is enabled

✗ DO NOT run `cargo run -- nodeA`
(It will conflict with the running service)

2. Start Additional Nodes (Separate Terminals)

Now extend the cluster.

Each node runs in its own terminal.

Terminal-2 → Node-B

```
sudo env "PATH=$PATH" "HOME=$HOME" cargo run -- nodeB
```

Leave this terminal running.

Terminal-3 → Node-C

```
sudo env "PATH=$PATH" "HOME=$HOME" cargo run -- nodeC
```

Leave this terminal running.

What this confirms

- Node-B and Node-C discover Node-A
- Attestation completes
- Same signed policy hash is enforced
- No port conflicts occur

3. Verify Kernel Enforcement (nftables)

nftables must be checked AFTER nodes are running

3.1. List all nftables tables

```
sudo nft list tables
```

Expected output includes:

```
table inet sgx_guardian
```

3.2. Inspect Guardian nftables rules

```
sudo nft list table inet sgx_guardian
```

Example structure:

```
table inet sgx_guardian {  
  chain input {  
    type filter hook input priority filter; policy drop;  
    iif "lo" accept  
    ct state established,related accept  
    tcp dport 22 accept  
    udp dport 5353 accept  
    tcp dport { 50051, 50052, 50053 } accept  
    ip saddr 10.0.0.0/24 tcp dport 443 accept  
  }  
}
```

4. Runtime Health Check (Optional)

```
sudo systemctl status sgx-guardian
```

Expected:

```
Active: active (running)
```

Confirms additional nodes did not disturb the primary service.

5. Runtime Logs (Operational Visibility)

Last 20 log entries

```
sudo journalctl -u sgx-guardian -n 20
```

6. Audit Logs (Read-Only Monitoring)

Follow live audit logs

```
sudo tail -f /var/log/sgx-guardian/audit-nodeA.log
```

Search policy-related events

```
sudo grep "Policy" /var/log/sgx-guardian/audit-nodeA.log
```

Search attestation events

```
sudo grep "Attestation" /var/log/sgx-guardian/audit-nodeA.log
```

Confirms:

- Enforcement decisions are auditable
- Logs are append-only
- Runtime is observable without mutation

7. Stop Primary Guardian (Clean Shutdown)

At the end of demo / testing:

```
sudo systemctl stop sgx-guardian
```

Node-B and Node-C can then be stopped manually with **Ctrl+C**.

FINAL Correct Runtime Order

```
systemctl start sgx-guardian
systemctl status sgx-guardian
cargo run -- nodeB
cargo run -- nodeC
nft list tables
nft list table inet sgx_guardian
journalctl / audit logs
systemctl stop sgx-guardian
```

Never run nodeA via cargo in this flow

CLOUD COMMAND

One command that enables all cloud-related behavior (HTTP)

```
sudo env "PATH=$PATH" "HOME=$HOME" \  
SGX_RUN MOCK_CLOUD=true \  
SGX_CLOUD_UPLINK_ENABLED=true \  
SGX_CLOUD_ENDPOINT=http://127.0.0.1:9443/uplink \  
cargo run -- nodeA
```

What this ONE command does (simple explanation)

This single command performs all required cloud-related actions using HTTP mock cloud:

- Starts Node-A
- Loads and enforces the signed policy
- Starts the embedded mock cloud server (HTTP)
- Sends outbound HTTP cloud uplink requests
- Starts the cloud heartbeat loop
- Handles cloud failures safely (node does not crash)
- Writes Cloud-related events to audit logs

No additional cloud commands are required beyond this

Post-Run Verification

1. Cloud-related audit logs (MOST IMPORTANT)

```
sudo grep "category":"Cloud" /var/log/sgx-guardian/audit-nodeA.log
```

You should see entries such as:

- Cloud uplink enabled
- Local mock cloud server started (DEV mode)
- Cloud uplink heartbeat started

(Failure or success entries — both are valid)

2. Live audit monitoring (optional)

```
sudo tail -f /var/log/sgx-guardian/audit-nodeA.log
```

3. Confirm the node did NOT crash

```
sudo grep "category":"Node" /var/log/sgx-guardian/audit-nodeA.log
```

Expected entry:

```
"Node started successfully"
```

Clean Reinstall, Verification & Audit Access

Scope

- This part documents the complete clean-cycle an admin may perform:
- Remove existing Guardian
- Install fresh .deb
- Verify service & enforcement
- Access audit logs safely
 - No policy signing
 - No key generation

- No cargo nodes

Entry Assumption

System already had SG-X Guardian installed before

Admin wants a clean reinstall (lab / demo / troubleshooting)

1. Stop Guardian (If Running)

```
sudo systemctl stop sgx-guardian
```

Verify:

```
sudo systemctl status sgx-guardian
```

Expected:

```
Active: inactive (dead)
```

2. Uninstall Guardian (.deb Purge)

This removes binaries + systemd unit

(configs/logs may remain by design)

```
sudo dpkg --purge sgx-guardian-client
```

Verify removal:

```
dpkg -l | grep sgx-guardian
```

Expected:

(no output)

3. Install Guardian Fresh (.deb)

```
sudo dpkg -i build/deb/sgx-guardian-client.deb
```

If dependencies complain:

```
sudo apt --fix-broken install
```

4. Verify systemd Service Exists

```
systemctl list-unit-files | grep sgx-guardian
```

Expected:

```
sgx-guardian.service enabled
```

5. Start Guardian (Production Mode)

```
sudo systemctl start sgx-guardian
```

Check status:

```
sudo systemctl status sgx-guardian
```

Expected (important line):

```
Active: active (running)
```

Confirms:

- Service started correctly
- Policy loaded automatically
- Enforcement pipeline initialized

6. Verify Kernel Enforcement (nftables)

Always check AFTER service is running

6.1. List nftables tables

```
sudo nft list tables
```

Expected:

```
table inet sgx_guardian
```

6.2. Inspect Guardian rules

```
sudo nft list table inet sgx_guardian
```

What this proves:

```
Default policy drop (Zero-Trust)
Only policy-allowed ports open
Enforcement at kernel level
```

7. Verify Runtime Logs (Sanity)

```
sudo journalctl -u sgx-guardian -n 20
```

Optional live follow:

```
sudo journalctl -u sgx-guardian -f
```

8. Audit Logs — Safe Access Control (IMPORTANT)

By default, audit logs are restricted.

Admin may temporarily grant read-only access.

READ-ONLY Access (SAFE)

```
sudo chmod o+rx /var/log/sgx-guardian
```

View logs:

```
sudo tail -f /var/log/sgx-guardian/audit-nodeA.log
```

Lock Back After Viewing (MANDATORY)

```
sudo chmod o-rx /var/log/sgx-guardian
```

- ✓ Restores audit integrity
 - ✓ Prevents accidental exposure
 - ✗ Edit Access (UNSAFE — NOT RECOMMENDED)
- For debugging ONLY — breaks audit guarantees

```
sudo chmod o+rw /var/log/sgx-guardian
```

OR (File-Level Only — Less Risky)

```
sudo chmod o+rw /var/log/sgx-guardian/audit-nodeA.log
```

Still not recommended in production.

9. Final Clean Operational Flow

```
systemctl stop sgx-guardian
dpkg --purge sgx-guardian-client
dpkg -i sgx-guardian-client.deb
systemctl start sgx-guardian
systemctl status sgx-guardian
nft list tables
nft list table inet sgx_guardian
journalctl -u sgx-guardian
chmod o+rx /var/log/sgx-guardian
chmod o-rx /var/log/sgx-guardian
```